

Unit-12: Introduction to Prolog

Unit-12 Roadmap: Introduction to Prolog

Topic No.	Main Topic	Subtopics to Cover	Depth Required	Importance
12.1	Introduction to Prolog	Definition, Logic Programming, AI language	Basic + Conceptual	Very Important
12.2	Features of Prolog	Declarative programming, rule-based nature	Basic	Important
12.3	Structure of Prolog Program	Facts, Rules, Queries	Basic + Syntax	Very Important
12.4	Facts in Prolog	Syntax, examples, database representation	Basic + Practical	Very Important
12.5	Rules in Prolog	IF-THEN logic, inference rules	Medium + Practical	Very Important
12.6	Queries / Goals	Asking questions in Prolog	Basic + Practical	Very Important
12.7	Converting English Sentences to Prolog	Natural language to facts/rules	Medium + Examples	Very Important
12.8	Prolog Terminology	Predicate, atom, clause, term, arity	Theory	Important
12.9	Variables in Prolog	Syntax, anonymous variables, scope	Basic + Examples	Very Important
12.10	Control Structures	AND, OR, conditional logic	Medium	Important
12.11	Arithmetic Operators	Addition, subtraction, multiplication, division, mod	Practical	Important
12.12	Matching / Unification	Pattern matching process	Medium + Conceptual	Very Important
12.13	Backtracking	Automatic search mechanism	Full Theory + Examples	Very Important
12.14	Cut Operator	Controlling backtracking using (!)	Medium	Important
12.15	Recursion in Prolog	Recursive predicates, recursive rules	Medium + Practical	Very Important
12.16	Lists in Prolog	List syntax, head-tail notation	Medium + Practical	Very Important
12.17	List Operations	Membership, concatenation, length	Practical	Important
12.18	Advantages of Prolog	AI reasoning, symbolic processing	Theory	Important
12.19	Limitations of Prolog	Execution speed, debugging difficulty	Theory	Optional
12.20	Applications of Prolog	Expert systems, NLP, theorem proving	Theory + Real-world	Important

12.1 Introduction to Prolog

What is Prolog?

Prolog stands for: **PRO**gramming in **LOG**ic

It is a:

- Logic programming language
- AI-oriented programming language

developed mainly for:

- Artificial Intelligence
- Expert Systems
- Natural Language Processing

Definition: Prolog is a declarative programming language based on formal logic and rule-based reasoning.

Main Idea of Prolog

Traditional programming languages focus on:

- HOW to solve a problem

Prolog focuses on:

- WHAT is true about the problem

This makes Prolog different from languages like:

- C
- C++
- Java
- Python

Nature of Prolog

Feature	Description
Declarative Language	Describes logic instead of steps
Logic-Based	Uses facts and rules
AI-Oriented	Designed for AI applications
Rule-Based	Knowledge stored as rules

History of Prolog

Year	Information
1972	Developed by Alain Colmerauer
Origin	France
Purpose	Artificial Intelligence research

Components of Prolog

A Prolog program mainly contains:

Component	Purpose
Facts	Store information
Rules	Define relationships
Queries	Ask questions

Example of Simple Prolog Fact

father(ram, shyam).

Meaning: Ram is the father of Shyam.

Example of Rule

grandfather(X,Y) :- father(X,Z), father(Z,Y).

Meaning:

X is grandfather of Y if X is father of Z and Z is father of Y.

Example of Query

?- father(ram, shyam).

Output:

Yes

Features of Prolog

Feature	Explanation
Automatic Backtracking	Searches automatically
Pattern Matching	Matches terms automatically
Symbolic Processing	Handles symbols efficiently
Database-Oriented	Stores knowledge as facts

Why Prolog is Important in AI?

Prolog is useful for:

- Knowledge representation
- Logical reasoning
- Expert systems
- NLP systems

because it naturally supports:

- Logic
- Inference
- Rule-based systems

Areas Where Prolog is Used

Application Area	Usage
Expert Systems	Medical diagnosis
NLP	Language understanding
Robotics	Decision making
Theorem Proving	Logical inference
AI Research	Knowledge systems

Advantages of Prolog

Advantage	Explanation
Easy knowledge representation	Facts and rules
Powerful reasoning	Built-in inference
Less coding	Declarative style

Limitations of Prolog

Limitation	Explanation
Slow execution	Compared to procedural languages
Difficult debugging	Backtracking complexity

One-Line Summary

Prolog is a logic programming language used in Artificial Intelligence for knowledge representation and reasoning.

12.2 Features of Prolog

Introduction

Prolog has several special features that make it suitable for:

- Artificial Intelligence
- Logical reasoning
- Expert systems

Unlike procedural languages, Prolog works using:

- Facts
- Rules
- Queries

Main Features of Prolog

No.	Feature
1	Declarative Programming
2	Logic-Based Language
3	Rule-Based Programming
4	Automatic Backtracking
5	Pattern Matching
6	Recursion Support
7	Dynamic Knowledge Base
8	Symbolic Processing
9	Built-in Search Mechanism
10	AI-Oriented Language

1. Declarative Programming

In declarative programming:

- Programmer specifies WHAT to solve
- Not HOW to solve

Example

```
father(ram, shyam).
```

This simply declares:

- A fact

without describing:

- Execution steps

2. Logic-Based Language

Prolog is based on:

- Predicate logic
- Formal logic

Programs are written using:

- Logical statements

3. Rule-Based Programming

Knowledge is represented using:

- Rules

Example

```
grandfather(X,Y) :- father(X,Z), father(Z,Y).
```

This defines logical relationship.

4. Automatic Backtracking

If one solution fails:

- Prolog automatically searches another solution

This process is called:

Backtracking

Example

likes(ram, mango).

likes(ram, apple).

Query:

?- likes(ram, X).

Possible outputs:

X = mango

X = apple

Prolog automatically searches all answers.

5. Pattern Matching (Unification)

Prolog automatically compares:

- Terms
- Variables
- Structures

This process is called:

Unification

Example

father(ram, shyam).

Query:

?- father(ram, X).

Result:

X = shyam

6. Recursion Support

Prolog supports recursion naturally.

Recursive rules help solve:

- Repeated subproblems

Example

ancestor(X,Y) :- parent(X,Y).

ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).

7. Dynamic Knowledge Base

Facts and rules can be:

- Added
- Removed
- Modified

during execution.

8. Symbolic Processing

Prolog works efficiently with:

- Symbols
- Strings
- Logical structures

This is useful in AI systems.

9. Built-in Search Mechanism

Prolog automatically performs:

- Searching
- Matching
- Inference

without explicit programming.

10. AI-Oriented Language

Prolog is specially designed for:

- Expert systems
- NLP
- Knowledge-based systems

Advantages of Prolog Features

Feature	Benefit
Backtracking	Automatic solution search
Declarative nature	Simpler coding
Logical reasoning	Better AI support

Comparison with Procedural Languages

Procedural Language	Prolog
Step-by-step execution	Logic-based execution
Focus on algorithm	Focus on facts and rules
Manual search	Automatic search

One-Line Summary

Prolog provides logic-based, declarative, and AI-oriented features such as backtracking, recursion, and pattern matching.

12.3 Structure of Prolog Program

Introduction

A Prolog program is made using:

- Facts
- Rules
- Queries

These components form the basic structure of Prolog programming.

Basic Structure of Prolog Program

Facts

Rules

Queries

1. Facts

Facts represent:

- Simple statements
- Known information

Facts are always true.

Syntax of Fact

predicate(argument1, argument2).

Example

father(ram, shyam).

Meaning:

Ram is the father of Shyam.

More Examples

likes(ram, mango).

student(rahul).

car(bmw).

2. Rules

Rules define:

- Relationships
- Conditions
- Logical inference

Syntax of Rule

head :- body.

Meaning:

IF body is true

THEN head is true

Example

grandfather(X,Y) :- father(X,Z), father(Z,Y).

Meaning:

X is grandfather of Y if X is father of Z and Z is father of Y.

Components of Rule

Part	Meaning
Head	Conclusion
Body	Condition

Another Example

brother(X,Y) :- father(Z,X), father(Z,Y).

3. Queries

Queries are used to:

- Ask questions
- Retrieve information

Queries begin with:

?-

Example Query

?- father(ram, shyam).

Output:

Yes

Query with Variable

?- father(ram, X).

Output:

X = shyam

Complete Prolog Program Example

father(ram, shyam).

father(shyam, mohan).

grandfather(X,Y) :- father(X,Z), father(Z,Y).

Query:

?- grandfather(ram, mohan).

Output:

Yes

Structure Visualization

Facts → Store Knowledge

Rules → Define Logic

Queries → Ask Questions

Program Execution in Prolog

Query

↓

Search Facts/Rules

↓

Matching (Unification)

↓

Backtracking if Needed

↓

Result

Important Syntax Rules

Rule	Description
Every statement ends with .	Mandatory
Variables start with capital letter	Example: X, Y
Predicates start with small letter	Example: father
Query starts with ?-	Used for questions

12.4 Facts in Prolog

Introduction

Facts are the simplest form of knowledge in Prolog.

A fact represents:

- Information
- Relationship

- Statement

that is always considered true.

Definition: A fact is a statement that describes a relationship between objects and is assumed to be true in Prolog.

Syntax of Fact

predicate(argument1, argument2).

Important Rules

Rule	Description
Ends with period .	Mandatory
Predicate starts with lowercase	Example: father
Constants start with lowercase	Example: ram
Variables start with uppercase	Example: X

Example of Facts

father(ram, shyam).

mother(sita, shyam).

likes(rahul, cricket).

student(amt).

Meaning of Facts

Fact	Meaning
father(ram, shyam).	Ram is father of Shyam
mother(sita, shyam).	Sita is mother of Shyam
likes(rahul, cricket).	Rahul likes cricket
student(amt).	Amit is a student

Predicate: The name before brackets is called:

Predicate

Example: father(ram, shyam).

Here:

- father is predicate

Arguments

Values inside brackets are called:

Arguments

Example: father(ram, shyam).

Arguments:

- ram
- shyam

Types of Facts

Type	Example
Single Argument Fact	student(amt).
Multiple Argument Fact	father(ram, shyam).

Family Relationship Example

father(ram, shyam).
father(shyam, mohan).
mother(gita, mohan).
These facts create a knowledge database.

Using Facts with Queries

Query Example

?- father(ram, shyam).

Output:

Yes

Another Query

?- mother(gita, mohan).

Output:

Yes

False Query Example

?- father(mohan, ram).

Output:

No

Query with Variables

?- father(ram, X).

Output:

X = shyam

Prolog searches facts automatically.

Knowledge Base

Collection of facts is called: **Knowledge Base**

Example:

father(ram, shyam).

father(shyam, mohan).

likes(rahul, cricket).

One-Line Summary

Facts in Prolog represent basic true statements stored in the knowledge base for logical reasoning.

12.5 Rules in Prolog

Introduction

Rules are used to define:

- Relationships
- Conditions
- Logical inference

Rules help Prolog derive:

- New information
from existing facts.

Definition: A rule is a logical statement that specifies a condition under which something is true.

General Structure of Rule

head :- body.

Meaning:

IF body is true

THEN head is true

Components of Rule

Component	Meaning
Head	Conclusion
Body	Condition

Example Rule

grandfather(X,Y) :- father(X,Z), father(Z,Y).

Meaning: X is grandfather of Y if X is father of Z and Z is father of Y.

Required Facts

father(ram, shyam).

father(shyam, mohan).

Query

?- grandfather(ram, mohan).

Output:

Yes

Another Rule Example

brother(X,Y) :- father(Z,X), father(Z,Y).

Meaning: X and Y are brothers if they have same father.

Rule with AND Operator

Comma , means: **AND**

Example

grandfather(X,Y) :- father(X,Z), father(Z,Y).

Conditions:

- X is father of Z
- AND
- Z is father of Y

Both must be true.

Rule with OR Operator

Semicolon ; means: **OR**

Example

parent(X,Y) :- father(X,Y); mother(X,Y).

Meaning:

X is parent of Y if X is father OR mother of Y.

Multiple Rules Example

parent(X,Y) :- father(X,Y).

parent(X,Y) :- mother(X,Y).

Rule Execution Process

Query



Search Rule



Check Conditions



Apply Matching



Return Result

Advantages of Rules

Advantage	Explanation
Logical reasoning	Derives new knowledge
Reduces repetition	Reusable logic
AI-friendly	Supports inference systems

Difference Between Facts and Rules

Facts	Rules
Direct information	Derived information
Always true	Condition-based
No conditions	Uses IF-THEN logic

Important Exam Points

Topic	Key Point
Rule syntax	head :- body
Comma	AND
Semicolon	OR
Purpose	Derive new information

One-Line Summary

Rules in Prolog define logical relationships and derive new information using conditions and inference.

12.6 Queries / Goals in Prolog

Introduction

Queries are used to:

- Ask questions
- Retrieve information
- Test facts and rules

Queries are also called:

Goals

Definition: A query is a request made to Prolog to determine whether a fact or rule is true.

Syntax of Query

?- query.

Example Query

?- father(ram, shyam).

Output:

Yes

Query with Variables

?- father(ram, X).

Output:

X = shyam

How Queries Work

When query is entered:

1. Prolog searches facts
2. Applies rules
3. Performs matching
4. Returns result

Example Database

father(ram, shyam).

father(shyam, mohan).

mother(sita, shyam).

Query Examples

1. Direct Query

?- mother(sita, shyam).

Output:

Yes

2. Query Returning Variable

?- father(X, mohan).

Output:

X = shyam

3. False Query

?- father(mohan, ram).

Output:

No

Query Using Rules

Facts

father(ram, shyam).

father(shyam, mohan).

Rule

grandfather(X,Y) :- father(X,Z), father(Z,Y).

Query

?- grandfather(ram, mohan).

Output:

Yes

Types of Queries

Type	Example
Simple Query	?- student(amt).
Variable Query	?- father(X,Y).
Rule-Based Query	?- grandfather(ram,mohan).

Goals in Prolog

A query is also called: **Goal**

Prolog tries to:

- Satisfy the goal

using:

- Facts
- Rules

Multiple Solutions

If multiple answers exist:

- Prolog uses backtracking

Example

likes(ram, mango).

likes(ram, apple).

Query:

?- likes(ram, X).

Outputs:

X = mango

X = apple

Query Execution Flow

Query

↓

Search Facts

↓

Search Rules

↓

Matching

↓

Backtracking if Needed

↓

Result

One-Line Summary

Queries (Goals) in Prolog are used to ask questions and retrieve information from facts and rules using logical inference.

12.7 Converting English Sentences to Prolog

Introduction

One of the most important features of Prolog is:
Converting natural language statements into logical facts and rules.
This process helps Prolog:

- Understand knowledge
- Perform reasoning
- Answer queries logically

Why Conversion is Needed?

Computers cannot directly understand:

- Human language

So English sentences are converted into:

- Facts
- Rules
- Predicates

for logical processing.

Basic Idea

English sentence: Ram is the father of Shyam.

Converted into Prolog:

father(ram, shyam).

General Conversion Method

English Part	Prolog Representation
Relationship	Predicate
Persons/Objects	Arguments
Logical condition	Rule

Step-by-Step Conversion Process

Step	Description
1	Identify relationship
2	Identify objects/persons
3	Create predicate
4	Write fact or rule

Converting Simple Statements into Facts

Example 1

English Sentence

Ram is the father of Shyam.

Prolog Fact

father(ram, shyam).

Example 2

English Sentence

Sita is the mother of Mohan.

Prolog Fact

mother(sita, mohan).

Example 3

English Sentence

Rahul likes cricket.

Prolog Fact

likes(rahul, cricket).

Converting Conditional Statements into Rules

Rules are used when sentence contains:

- IF
- THEN
- Conditions
- Relationships

Example 1: Grandfather Rule

English Sentence

X is grandfather of Y if X is father of Z and Z is father of Y.

Prolog Rule

grandfather(X,Y) :- father(X,Z), father(Z,Y).

Understanding the Rule

Part	Meaning
grandfather(X,Y)	Conclusion
father(X,Z)	Condition 1
father(Z,Y)	Condition 2

Example 2: Parent Rule

English Sentence

X is parent of Y if X is father of Y or X is mother of Y.

Prolog Rule

parent(X,Y) :- father(X,Y); mother(X,Y).

AND and OR in Prolog

English Word	Prolog Symbol
AND	,
OR	;

Example Using AND

English Statement : A person is grandfather if he is father of parent and parent is father of child.

Prolog: grandfather(X,Y) :- father(X,Z), father(Z,Y).

Example Using OR

English Statement

A person is parent if he is father or mother.

Prolog

parent(X,Y) :- father(X,Y); mother(X,Y).

Family Relationship Example

English Statements

Ram is father of Shyam.

Shyam is father of Mohan.

Prolog Facts

father(ram, shyam).

father(shyam, mohan).

English Rule

A grandfather is father of father.

Prolog Rule

grandfather(X,Y) :- father(X,Z), father(Z,Y).

Query

?- grandfather(ram, mohan).

Output:

Yes

Another Example: Brother Relationship

English Statement

X and Y are brothers if they have same father.

Prolog Rule

brother(X,Y) :- father(Z,X), father(Z,Y).

Important Syntax Rules

Rule	Example
Statements end with .	father(ram, shyam).
Variables use capital letters	X, Y, Z
Predicates use lowercase	father
Query starts with ?-	?- father(ram,X).

Translation Examples Table

English Sentence	Prolog Representation
Ram is student	student(ram).
Ram likes mango	likes(ram, mango).
Sita is mother of Mohan	mother(sita, mohan).
X is parent if X is father or mother	parent(X,Y) :- father(X,Y); mother(X,Y).

Advantages of English-to-Prolog Conversion

Advantage	Explanation
Easy knowledge representation	Natural statements become logic
Supports AI reasoning	Logical inference possible
Simple rule creation	Human-readable logic

Common Mistakes

Mistake	Correction
Missing period	Add .
Variable in lowercase	Use uppercase
Predicate in uppercase	Use lowercase

Important Exam Points

Topic	Key Point
Facts	Direct statements
Rules	Conditional statements
AND operator	,
OR operator	;
Variables	Use uppercase

One-Line Summary

English sentences are converted into Prolog facts and rules to represent knowledge logically for reasoning and inference.

12.8 Prolog Terminology

Introduction

Prolog uses several special terms related to:

- Logic programming
- Knowledge representation
- Rule-based systems

Understanding these terms is essential for writing and understanding Prolog programs.

Important Prolog Terminologies

Term	Meaning
Predicate	Relation/function name
Atom	Constant symbolic value
Variable	Placeholder for unknown value
Term	Basic data object
Clause	Fact or Rule
Fact	True statement
Rule	Conditional statement
Query / Goal	Question asked to Prolog
Arity	Number of arguments
Knowledge Base	Collection of facts and rules

1. Predicate

A predicate represents:

- Relationship
- Property
- Action

Example

father(ram, shyam).

Here:

- father is predicate

2. Atom

Atoms are:

- Constant names

- Symbolic values

Usually written in:

- Lowercase

Example

ram

mango

cricket

These are atoms.

3. Variable

Variables represent:

- Unknown values

Variables always start with:

- Capital letter

Example

X

Y

Student

Example in Query

?- father(ram, X).

Here:

- X is variable

4. Term

Everything in Prolog is a term.

Types of terms:

- Atom
- Variable
- Structure

Example

father(ram, shyam)

This complete expression is a term.

5. Clause

A clause is:

- Fact
OR
- Rule

Fact Clause

student(amt).

Rule Clause

parent(X,Y) :- father(X,Y).

6. Fact

Fact represents:

- True statement

Example

likes(rahul, cricket).

7. Rule

Rule represents:

- Conditional logic

Example

grandfather(X,Y) :- father(X,Z), father(Z,Y).

8. Query / Goal

Query asks questions to Prolog.

Example

?- likes(rahul, cricket).

9. Arity

Arity means:

Number of arguments in predicate

Example

father(ram, shyam).

Predicate:

- father

Arguments:

- ram
- shyam

Arity:

- 2

Written as:

father/2

More Examples of Arity

Predicate	Arity
student(amt)	1
father(ram, shyam)	2
sum(X,Y,Z)	3

10. Knowledge Base

Collection of:

- Facts
- Rules

is called:

Knowledge Base

Example

father(ram, shyam).

mother(sita, shyam).
parent(X,Y) :- father(X,Y).

Summary Table

Terminology	Example
Predicate	father
Atom	ram
Variable	X
Fact	student(amt).
Rule	parent(X,Y):-father(X,Y).
Query	?- father(ram,X).
Arity	father/2

Important Exam Points

Topic	Key Point
Atom	Lowercase constant
Variable	Starts with capital
Predicate	Relation name
Arity	Number of arguments

One-Line Summary

Prolog terminology defines the basic logical components used in Prolog programming and reasoning.

12.9 Variables in Prolog

Introduction

Variables are used in Prolog to represent:

- Unknown values
- Temporary values
- Generalized relationships

Variables make Prolog programs:

- Flexible
- Reusable
- Dynamic

Definition A variable in Prolog is a placeholder that can match different values during execution.

Rules for Variables

Rule	Description
Starts with capital letter	Example: X, Y
Can contain letters/numbers	Example: Student1
Represents unknown value	Assigned during matching

Examples of Variables

X

Y

Person

Student1

Example Using Variables

father(ram, shyam).

Query:

?- father(ram, X).

Output:

X = shyam

Here:

- X gets matched automatically.

Purpose of Variables

Variables are used for:

- Searching
- Matching
- Generalization
- Query processing

Variables in Rules

Variables make rules generic.

Example

parent(X,Y) :- father(X,Y).

Meaning:

For any X and Y,

if X is father of Y,

then X is parent of Y.

Anonymous Variable

Anonymous variable is written using:

—

It means:

- Value is not important

Example

father(ram, _).

Meaning:

- Ram is father of someone,
but we do not care who.

Multiple Variables Example

likes(rahul, cricket).

likes(amit, football).

Query:

?- likes(X, Y).

Output:

X = rahul, Y = cricket

X = amit, Y = football

Variable Matching Process

Variable

↓

Search Facts/Rules



Match Values



Return Result

Scope of Variables

Variables are:

- Local to a rule/query

Same variable name inside one rule:

- Represents same value

Example

same(X,X).

Query:

?- same(5,5).

Output:

Yes

Advantages of Variables

Advantage	Explanation
Flexible programming	Generalized logic
Easy searching	Automatic matching
Reusable rules	Dynamic reasoning

Important Exam Points

Topic	Key Point
Variable	Unknown value holder
Naming rule	Starts with capital
Anonymous variable	_
Main use	Matching and queries

One-Line Summary

Variables in Prolog represent unknown values and help perform matching, querying, and logical reasoning dynamically.

12.10 Control Structures in Prolog

Introduction

Control structures determine:

- Flow of execution
- Logical conditions
- Decision making

Prolog uses logical operators instead of traditional loops and conditions.

Main Control Structures in Prolog

Control Structure	Symbol
AND	,
OR	;
NOT	\+

IF-THEN	->
Cut	!

1. AND Operator

AND means:

- All conditions must be true

Represented by:

,

Example

grandfather(X,Y) :- father(X,Z), father(Z,Y).

Conditions:

- X is father of Z
- AND
- Z is father of Y

2. OR Operator

OR means:

- At least one condition should be true

Represented by: ;

Example

parent(X,Y) :- father(X,Y); mother(X,Y).

Meaning:

- X is parent if father OR mother.

3. NOT Operator

NOT checks:

- Failure of condition

Represented by: \+

Example

not_parent(X,Y) :- \+ parent(X,Y).

Meaning:

- X is not parent of Y.

4. IF-THEN Structure

Syntax:

Condition -> Action.

Example

greater(X,Y) :- X > Y -> write('X is greater').

5. Cut Operator

Cut operator: !

Purpose:

- Stops unnecessary backtracking

Example

max(X,Y,X) :- X > Y, !.

max(X,Y,Y).

Execution Flow in Prolog

Query



Check Conditions



Apply AND/OR/NOT



Backtracking if Needed



Result

Difference from Procedural Languages

Procedural Languages	Prolog
if-else	Rules and logic
loops	Recursion
manual control	Automatic inference

Advantages of Prolog Control Structures

Advantage	Explanation
Simple logical flow	Easy reasoning
AI-friendly	Supports inference
Automatic searching	Built-in backtracking

Important Exam Points

Topic	Symbol
AND	,
OR	;
NOT	\+
Cut	!

One-Line Summary

Control structures in Prolog use logical operators to control execution flow and reasoning.

12.11 Arithmetic Operators in Prolog

Introduction

Prolog supports arithmetic operations for:

- Calculations
- Comparisons
- Mathematical processing

Arithmetic operations are mainly used with:

- Variables
- Numeric expressions

Common Arithmetic Operators

Operator	Meaning
+	Addition

-	Subtraction
*	Multiplication
/	Division
mod	Modulus
**	Power

Arithmetic Evaluation Operator

Prolog uses: is
to evaluate arithmetic expressions.

Syntax

Variable is Expression.

Example: Addition

X is 5 + 3.

Output:

X = 8

Example: Subtraction

X is 10 - 4.

Output:

X = 6

Example: Multiplication

X is 6 * 2.

Output:

X = 12

Example: Division

X is 10 / 2.

Output:

X = 5

Example: Modulus

X is 10 mod 3.

Output:

X = 1

Example: Power

X is 2 ** 3.

Output:

X = 8

Comparison Operators

Operator	Meaning
>	Greater than
<	Less than
>=	Greater or equal
=<	Less or equal

==	Equal
=\=	Not equal

Example

5 > 3.

Output:

Yes

Using Arithmetic in Rules

Example

square(X,Y) :- Y is X * X.

Query:

?- square(5,Y).

Output:

Y = 25

Example: Maximum of Two Numbers

max(X,Y,X) :- X > Y.

max(X,Y,Y) :- Y >= X.

Arithmetic Execution Process

Expression

↓

Evaluate using 'is'

↓

Assign value to variable

↓

Return result

Important Notes

Note	Explanation
is evaluates expression	Mandatory for calculations
Variables must be instantiated	Before arithmetic use
Arithmetic differs from matching	= is not arithmetic evaluation

Difference Between = and is

Operator	Purpose
=	Matching
is	Arithmetic evaluation

Example

X = 5 + 3.

Result: X = 5 + 3

No calculation occurs.

X is 5 + 3.

Result: X = 8

Calculation occurs.

Advantages of Arithmetic Operators

Advantage	Explanation
Mathematical computation	Easy calculations
Supports AI logic	Numeric reasoning
Useful in recursion	Recursive calculations

Important Exam Points

Topic	Key Point
is operator	Arithmetic evaluation
= operator	Matching only
mod	Remainder
Comparison operators	Numeric comparison

One-Line Summary

Arithmetic operators in Prolog perform mathematical calculations and comparisons using the is evaluation operator.

12.12 Matching / Unification in Prolog

Introduction

Matching is one of the most important concepts in Prolog.

Prolog solves problems mainly through:

- Matching
- Pattern comparison
- Variable binding

This process is called: **Unification**

Definition: Unification is the process of making two terms equal by finding suitable values for variables.

Purpose of Unification

Unification helps Prolog:

- Match queries with facts
- Match rules with conditions
- Assign values to variables
- Perform logical reasoning

Basic Idea

Suppose Prolog has fact: father(ram, shyam).

Query:

?- father(ram, X).

During unification: X = shyam

Variable X gets matched automatically.

Types of Matching

Type	Example
Constant Matching	ram = ram
Variable Matching	X = ram
Structure Matching	father(X,Y)

1. Constant Matching

Two constants match only if:

- They are exactly same

Example

ram = ram.

Output: Yes

Non-Matching Example

ram = shyam.

Output: No

2. Variable Matching

Variables can match:

- Constants
- Other variables
- Structures

Example

X = ram.

Output: X = ram

Another Example

X = Y.

Both variables become linked.

3. Structure Matching

Structures match if:

- Predicate names are same
- Arity is same
- Arguments match

Example

father(X,Y) = father(ram, shyam).

Result:

X = ram

Y = shyam

Unification Rules

Two terms unify if:

Condition	Requirement
Predicate names	Must be same
Arity	Must be same
Arguments	Must match

Example of Failed Unification

father(X,Y) = mother(ram, shyam).

Output: No

because:

- Predicate names differ

Matching in Query Processing

Facts

likes(ram, mango).

likes(shyam, apple).

Query

?- likes(X, mango).

Matching Result

X = ram

Unification Process Flow

Query



Search Facts/Rules



Compare Terms



Assign Variable Values



Return Result

Importance of Unification

Importance	Explanation
Core execution mechanism	Main reasoning process
Variable binding	Assigns values automatically
Knowledge retrieval	Finds matching facts

Advantages of Unification

Advantage	Explanation
Automatic matching	Less manual coding
Powerful reasoning	Supports AI systems
Flexible querying	Dynamic variable assignment

Important Exam Points

Topic	Key Point
Unification	Matching process
Variable matching	Assigns values
Structure matching	Predicate + arguments
Failure condition	Different predicate or arity

One-Line Summary

Unification is the process of matching terms and assigning values to variables for logical reasoning in Prolog.

12.13 Backtracking in Prolog

Introduction

Backtracking is one of the most powerful features of Prolog.

It allows Prolog to:

- Automatically search for solutions
- Try alternative paths
- Find multiple answers

Definition

Backtracking is the process in which Prolog automatically returns to previous points to search for alternative solutions when a path fails.

Why Backtracking is Needed?

In logical problems:

- Multiple solutions may exist
- Some paths may fail

Backtracking helps Prolog:

- Retry other possibilities
- Continue searching automatically

Basic Working Idea

Try Solution



Success? → Return Result



Failure



Go Back (Backtrack)



Try Another Solution

Example Knowledge Base

likes(ram, mango).

likes(ram, apple).

likes(ram, banana).

Query

?- likes(ram, X).

Execution Process

Prolog first finds:

X = mango

If user asks for more solutions (;):

Prolog backtracks and returns:

X = apple

X = banana

Step-by-Step Backtracking Process

Step	Action
1	Match first fact

2	Return first answer
3	Search for more answers
4	Go back to previous choice
5	Try next fact

Example with Failure

Facts

father(ram, shyam).

father(shyam, mohan).

Query

?- father(mohan, X).

Result

No

Prolog:

- Searches all facts
- Finds no match
- Stops execution

Choice Point

Whenever multiple possibilities exist,

Prolog creates: **Choice Point**

Used for:

- Future backtracking

Example of Choice Point

likes(ram, mango).

likes(ram, apple).

Two possible matches create:

- Choice point

Backtracking in Rules

Facts

father(ram, shyam).

father(shyam, mohan).

father(ram, amit).

Rule

grandfather(X,Y) :- father(X,Z), father(Z,Y).

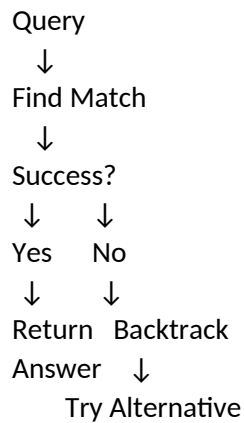
Query

?- grandfather(ram, Y).

Prolog:

- Tries different values of Z
- Uses backtracking if needed

Execution Flow



Advantages of Backtracking

Advantage	Explanation
Automatic search	No manual search coding
Multiple solutions	Finds all possible answers
Simplifies logic programming	Easier AI reasoning

Disadvantages of Backtracking

Disadvantage	Explanation
Extra computation	May search unnecessary paths
Slow execution	Large search space increases time

Applications of Backtracking

Application	Usage
Puzzle solving	N-Queens, Sudoku
AI search	Path finding
NLP	Sentence parsing

Important Exam Points

Topic	Key Point
Backtracking	Automatic retry mechanism
Choice point	Stores alternatives
; symbol	Requests next solution
Main purpose	Find multiple answers

One-Line Summary

Backtracking is Prolog's automatic mechanism for searching alternative solutions when previous attempts fail.

12.14 Cut Operator in Prolog

Introduction

Backtracking is useful, but sometimes:

- Unnecessary searching occurs
- Extra solutions are generated
- Execution becomes slow

To control this, Prolog uses: **Cut Operator**

Definition: Cut operator (!) is used to stop unnecessary backtracking in Prolog.

Symbol of Cut: !

Purpose of Cut Operator

Cut helps:

- Improve efficiency
- Reduce unnecessary searching
- Prevent unwanted solutions

Basic Idea

When Prolog reaches cut (!):

- Previous choices are discarded
- Backtracking before cut becomes impossible

Example Without Cut

`max(X,Y,X) :- X > Y.`

`max(X,Y,Y).`

Query

`?- max(5,3,Z).`

Possible behavior:

- Prolog may still check second rule unnecessarily.

Example With Cut

`max(X,Y,X) :- X > Y, !.`

`max(X,Y,Y).`

Query

`?- max(5,3,Z).`

Output: `Z = 5`

After cut:

- Prolog stops checking other rules.

Working of Cut

Condition True

↓

Encounter !

↓

Stop Backtracking

↓

Return Result

Another Example

`grade(Marks, distinction) :- Marks >= 75, !.`

`grade(Marks, firstclass) :- Marks >= 60.`

Query

`?- grade(80, X).`

Output: `X = distinction`

Cut prevents checking second rule.

Types of Cut

Type	Description
Green Cut	Improves efficiency only
Red Cut	Changes program logic

1. Green Cut

- Does not change meaning
- Only improves performance

2. Red Cut

- Changes logic/output
- Must be used carefully

Advantages of Cut

Advantage	Explanation
Faster execution	Stops unnecessary search
Better efficiency	Reduces computation
Controls backtracking	Cleaner output

Disadvantages of Cut

Disadvantage	Explanation
Difficult debugging	Flow becomes complex
Can affect logic	Improper use causes errors

Cut vs Backtracking

Backtracking	Cut
Searches alternatives	Stops alternatives
Automatic search	Search control
More solutions	Restricted solutions

Applications of Cut

Application	Usage
Decision making	First valid result
Optimization	Faster execution
Rule control	Avoid duplicate solutions

Important Exam Points

Topic	Key Point
Symbol	!
Purpose	Stop backtracking
Green cut	Efficiency improvement
Red cut	Logic-changing cut

One-Line Summary

The cut operator (!) controls Prolog backtracking by preventing unnecessary alternative searches.

12.15 Recursion in Prolog

Introduction

Recursion is one of the most important concepts in Prolog.
Since Prolog does not use traditional loops like:

- for
- while

repetition is mainly achieved using:

Recursion

Definition

Recursion is a technique in which a rule calls itself repeatedly until a stopping condition is reached.

Why Recursion is Needed?

Recursion is used for:

- Repeated operations
- Problem decomposition
- Searching
- Mathematical calculations
- List processing

Basic Idea of Recursion

A recursive rule has:

1. Base Condition
2. Recursive Condition

Structure of Recursive Rule

base_case.

recursive_case :- condition, recursive_call.

Components of Recursion

Component	Purpose
Base Case	Stops recursion
Recursive Case	Calls itself again

Example 1: Ancestor Relationship

Facts

```
parent(ram, shyam).  
parent(shyam, mohan).  
parent(mohan, amit).
```

Recursive Rules

```
ancestor(X,Y) :- parent(X,Y).  
ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).
```

Explanation

Base Rule

```
ancestor(X,Y) :- parent(X,Y).
```

Meaning:

- Direct parent is ancestor.

Recursive Rule

`ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).`

Meaning:

- X is ancestor of Y
if:
- X is parent of Z
AND
- Z is ancestor of Y

Query Example

?- `ancestor(ram, amit).`

Output:

Yes

Recursion Execution Flow

`ancestor(ram, amit)`

↓

`parent(ram, shyam)`

↓

`ancestor(shyam, amit)`

↓

`parent(shyam, mohan)`

↓

`ancestor(mohan, amit)`

↓

`parent(mohan, amit)`

↓

Success

Example 2: Factorial Program

Mathematical factorial:

$$n! = n \times (n - 1)!$$

Prolog Program

`factorial(0,1).`

`factorial(N,F) :-`

`N > 0,`

`N1 is N - 1,`

`factorial(N1,F1),`

`F is N * F1.`

Query

?- `factorial(5,F).`

Output:

F = 120

Step-by-Step Factorial Working

```

factorial(5)
  ↓
5 × factorial(4)
  ↓
4 × factorial(3)
  ↓
3 × factorial(2)
  ↓
2 × factorial(1)
  ↓
1 × factorial(0)
  ↓
1

```

Importance of Base Case

Without base case:

- Recursion never stops
- Infinite recursion occurs

Infinite Recursion Example

loop :- loop.

This creates:

- Endless recursive calls

Applications of Recursion in Prolog

Application	Usage
Family relationships	Ancestor problems
Mathematical calculations	Factorial, Fibonacci
AI search	Tree traversal
List processing	Recursive list operations

Recursion vs Iteration

Recursion	Iteration
Function/rule calls itself	Uses loops
Main repetition method in Prolog	Used in procedural languages
Elegant logic	Faster execution

Important Exam Points

Topic	Key Point
Recursion	Rule calling itself
Base case	Stops recursion
Recursive case	Repeats processing
Main use	Repetition in Prolog

One-Line Summary

Recursion in Prolog is a technique where rules call themselves repeatedly to solve problems step by step until a base condition is reached.

12.16 Lists in Prolog

Introduction

A list is one of the most important data structures in Prolog.

Lists are used to store:

- Multiple values
- Sequences of elements
- Collections of data

Lists are heavily used in:

- AI programming
- Recursive processing
- Symbolic computation

Definition

A list is an ordered collection of elements enclosed within square brackets [].

Syntax of List

[element1, element2, element3]

Examples of Lists

[1,2,3,4]

[apple, mango, banana]

[a,b,c]

Empty List

An empty list contains no elements.

Syntax:

[]

Characteristics of Lists

Characteristic	Explanation
Ordered collection	Elements maintain order
Dynamic size	Can grow/shrink
Supports recursion	Easy recursive processing

Types of List Elements

A list may contain:

- Numbers
- Atoms
- Variables
- Other lists

Example

[1, apple, X, [a,b]]

Head and Tail of List

Prolog divides list into:

1. Head
2. Tail

Structure

[Head | Tail]

Example

[a,b,c,d]

can be represented as:

[a | [b,c,d]]

Understanding Head and Tail

Part	Value
Head	a
Tail	[b,c,d]

Another Example

[1,2,3,4]

Representation:

[1 | [2,3,4]]

Accessing Head and Tail

Query

?- [H|T] = [1,2,3,4].

Output:

H = 1

T = [2,3,4]

List Matching

Lists are matched using:

- Unification

Example

?- [X,Y] = [apple,mango].

Output:

X = apple

Y = mango

Nested Lists

Lists can contain:

- Other lists

Example

[[a,b],[c,d]]

Recursive Nature of Lists

Lists are naturally recursive.

A list is:

- Empty list
OR
- Head + Tail

Importance of Lists

Importance	Explanation
Data storage	Store sequences
Recursive processing	Easy recursion
AI programming	Symbolic data handling

Applications of Lists

Application	Usage
AI systems	Knowledge representation
NLP	Sentence processing
Search algorithms	State storage

Important Exam Points

Topic	Key Point
List syntax	[]
Empty list	[]
Head-tail notation	`[H
Nested lists	Lists inside lists

One-Line Summary

Lists in Prolog are ordered collections of elements used for recursive processing and symbolic data representation.

12.17 List Operations in Prolog

Introduction

Prolog provides powerful operations for:

- Accessing lists
- Searching elements
- Combining lists
- Finding length

Most list operations are implemented using:

Recursion

Common List Operations

Operation	Purpose
Membership	Check element existence
Concatenation	Join lists
Length	Count elements
Append	Add element/list
Delete	Remove element
Reverse	Reverse order

1. Membership Operation

Checks whether an element exists in list.

Built-in Predicate

`member(Element, List).`

Example

?- member(apple, [apple,mango,banana]).

Output:

Yes

Another Example

?- member(X, [1,2,3]).

Output:

X = 1

X = 2

X = 3

using backtracking.

2. Concatenation of Lists

Concatenation joins two lists.

Built-in Predicate

append(List1, List2, Result).

Example

?- append([1,2], [3,4], X).

Output:

X = [1,2,3,4]

3. Length of List

Finds number of elements.

Built-in Predicate

length(List, Length).

Example

?- length([a,b,c,d], X).

Output:

X = 4

4. Adding Element to List

Element can be added using:

- Append
- Head-tail notation

Example

?- append([1,2,3], [4], X).

Output:

X = [1,2,3,4]

5. Delete Element from List

Deletes specified element.

Example Predicate

delete(X,[X|T],T).

delete(X,[H|T],[H|T1]) :-
delete(X,T,T1).

Query

?- delete(2,[1,2,3],X).

Output:

X = [1,3]

6. Reverse List

Reverses order of elements.

Built-in Predicate

reverse(List, ReversedList).

Example

?- reverse([1,2,3], X).

Output:

X = [3,2,1]

Recursive List Processing Example

Sum of List Elements

sum([],0).

sum([H|T],S) :-

sum(T,S1),

S is H + S1.

Query

?- sum([1,2,3],X).

Output:

X = 6

Working of Recursive List Processing

List

↓

Head + Tail

↓

Process Head

↓

Recursive call on Tail

↓

Base Case

Applications of List Operations

Application	Usage
NLP	Sentence token storage
AI search	State representation
Data processing	Collection handling

Important Exam Points

Topic	Key Point
member()	Membership checking
append()	Concatenation
length()	Count elements
`[H	T]`

One-Line Summary

List operations in Prolog are used to manipulate and process lists using recursion and built-in predicates.

12.18 Advantages of Prolog

Introduction

Prolog is widely used in Artificial Intelligence because of its:

- Logical reasoning capability
- Declarative nature
- Built-in search mechanisms

It provides several advantages over traditional programming languages for AI applications.

Major Advantages of Prolog

No.	Advantage	Explanation
1	Simple Knowledge Representation	Uses facts and rules
2	Declarative Programming	Focuses on WHAT, not HOW
3	Automatic Backtracking	Searches solutions automatically
4	Powerful Pattern Matching	Built-in unification mechanism
5	Easy Rule-Based Programming	Supports logical inference
6	Recursion Support	Natural recursive processing
7	AI-Oriented Language	Designed for AI applications
8	Less Coding Required	Logical statements reduce code size
9	Dynamic Database	Facts/rules can be modified
10	Efficient Symbolic Processing	Works well with symbolic information

Applications Benefiting from Prolog

Application	Benefit
Expert Systems	Rule-based reasoning
NLP	Symbolic language processing
AI Search	Automatic inference
Robotics	Decision making

One-Line Summary

Prolog provides powerful logical reasoning, automatic backtracking, and declarative programming features for AI applications.

12.19 Limitations of Prolog

Introduction

Although Prolog is powerful for AI and logical reasoning, it also has several limitations.

These limitations may affect:

- Performance

- Scalability
- Ease of programming

Major Limitations of Prolog

No.	Limitation	Explanation
1	Slow Execution	Backtracking increases runtime
2	Difficult Debugging	Logic tracing is complex
3	High Memory Usage	Recursive calls consume memory
4	Less Efficient for Numerical Computation	Not ideal for heavy mathematics
5	Complex Backtracking Behavior	Hard to predict execution flow
6	Limited Procedural Control	Less control over execution steps
7	Infinite Recursion Risk	Missing base case causes loops
8	Not Suitable for Large Applications	Scalability issues

12.20 Applications of Prolog

Introduction

Prolog is mainly used in:

- Artificial Intelligence
- Knowledge-based systems
- Logical reasoning applications

because of its:

- Rule-based structure
- Inference mechanism
- Symbolic processing capability

Major Applications of Prolog

Field	Application
Artificial Intelligence	Logical reasoning
Expert Systems	Decision support
Natural Language Processing	Language understanding
Robotics	Intelligent control
Theorem Proving	Mathematical logic
Database Systems	Query processing
Machine Learning	Symbolic learning
Game AI	Rule-based decision making

1. Expert Systems

One of the biggest applications of Prolog.

Expert systems use:

- Facts
- Rules
- Inference

to simulate human expertise.

Example

Medical diagnosis system:

- Symptoms stored as facts
- Diagnosis stored as rules

2. Natural Language Processing (NLP)

Prolog is used in:

- Sentence parsing
- Language understanding
- Grammar checking

because it supports:

- Symbolic processing
- Pattern matching

Example

Analyzing sentence structure:

Subject → Verb → Object

3. Robotics

Robots use Prolog for:

- Decision making
- Planning
- Logical control

Example

Robot path planning:

- Facts → Environment data
- Rules → Movement decisions

4. Theorem Proving

Prolog is useful for:

- Mathematical proofs
- Logical verification

because it supports:

- Predicate logic
- Inference mechanisms

5. Database Query Systems

Prolog can act like:

- Intelligent database system

using:

- Logical queries
- Rule-based retrieval

Example Query

?- father(ram, X).

6. Machine Learning

Prolog supports:

- Symbolic machine learning
- Rule generation
- Knowledge inference

7. Game AI

Used for:

- Rule-based game logic
- Intelligent decision making

Example

Chess move reasoning:

- Facts → Board state
- Rules → Legal moves

8. Knowledge Representation

Knowledge is stored using:

- Facts
- Rules
- Relationships

making Prolog useful in:

- Knowledge-based AI systems

Real-World Applications

Real-World Area	Usage
Medical Diagnosis	Expert systems
Chatbots	NLP
Legal Systems	Rule-based reasoning
Automated Planning	AI decision systems

Why Prolog is Suitable for AI?

Feature	AI Benefit
Backtracking	Automatic search
Unification	Pattern matching
Declarative nature	Easy knowledge representation
Rules	Logical inference